

PROYECTO FINAL

“Attor - Editorial de obras multiformato”

Alumnos:

- Juan Carlos Malanco Rodriguez
- Isabel Kyra Romero Borunda
- Jocelyn Flores Gutiérrez

Maestro: Adalberto Emmanuel Rojas Perea

Materia: Desarrollo de Aplicaciones Móviles

Fecha: 29/09/2026

● Índice de contenidos

● Índice de contenidos.....	1
● Planteamiento.....	3
● Proyecto.....	3
● Objetivo.....	3
● Justificación.....	3
● Público objetivo.....	3
● Necesidades detectadas.....	4
● Diseño.....	5
● Boceto / Wireframes.....	5
● Flujo de usuarios.....	6
● Identidad visual.....	9
● Paleta de colores.....	10
Elección funcional:.....	10
Estética narrativa:.....	10
Usabilidad:.....	10
● Prototipo inicial de Figma.....	11
● Modificaciones al prototipo de Figma.....	12
● Primera versión del prototipo funcional en Android Studio.....	13
● Versiones avanzadas del prototipo en Android Studio.....	13
● Funcionalidades.....	15
● Implementación de accesibilidad e inclusión.....	16
● Implementación de interacciones, validaciones y manejo de eventos.....	17
● Tecnologías y herramientas utilizadas.....	17
● Implementación de la persistencia de datos.....	18
● Pruebas unitarias.....	20
● Uso de IA.....	21

Desarrollar y presentar una aplicación móvil funcional e integral, aplicando los conceptos estudiados durante el curso, desde el diseño UX/UI hasta la integración de datos, accesibilidad, optimización y preparación para su distribución.

Repositorio en GitHub:

<https://github.com/Jcmalanco/Proyecto-Apps>

Diseño en Figma:

<https://www.figma.com/design/ydOd5T7U6uAWI7MAgOylOw/Attor?node-id=0-1&t=6PMopMpxpyF1M9E1-0>

Landing page:

https://isabelkyrom.github.io/Attor_landing_page

Video de prueba de la aplicación:

<https://drive.google.com/drive/folders/1IBGpM94f6WGZhjlfXSpXJivjBLfsZ9X?usp=sharing>

● Planteamiento

● Proyecto

Plataforma editorial de publicación y consumo de obras multiformato (libros, cómics, audiolibros, videos, etc.) en un mismo lugar.

● Objetivo

La aplicación busca facilitar la tarea de seguir tus historias favoritas reuniend obras de distintos formatos en un mismo lugar, albergando tanto a lectores como autores.

● Justificación

Actualmente no se cuenta con plataformas que se especialicen en reunir distintos formatos, sino que cada plataforma o aplicación gira en torno a un formato en particular, algunos ejemplos serían: Wattpad, Inkitt y AO3, que manejan libros y novelas; Webtoon, Tapas, Bilibili, que son de webcomics; y hay historias que se reparten entre redes sociales como Instagram y Youtube, e incluso plataformas de música para historias que son musicales (como Epic y Hamilton) o podcast (como Caso 63).

Por esa falta de una plataforma que reúna varios formatos, los autores a menudo se decantan por repartir su trabajo en varios lugares y colocar los links en sus perfiles de redes sociales para que los fans puedan consumir el contenido completo, pero es común que todo el proceso y material repartido son confusos y a menudo los fans no conocen la existencia de alguno si no se pone un gran esfuerzo en seguir la historia.

La aplicación pretende facilitar la tarea de seguir tus historias favoritas con mayor comodidad, menor incertidumbre y menor deserción de lectores.

● Público objetivo

- **Autores:** Autores de cómics, novelas, musicales, audiolibros, animación, etc. y autores con obras en distintos formatos.
- **Storyers:** Lectores y consumidores de las obras.

● Necesidades detectadas

Autores:

- Plataforma que soporte varios formatos (texto, video, pdf, audio, imagen, etc.)
- Comodidad de publicación y edición de la obras
- Opción de publicación y despublicación de las obras
- Acceso a las obras publicadas

Lectores:

- Comodidad de lectura
- Interacción con la obra (comentarios, calificación, guardar)
- Recomendaciones de obras y tendencias
- Biblioteca de obras guardadas
- Estatus de la lectura (porcentaje o etiquetas)

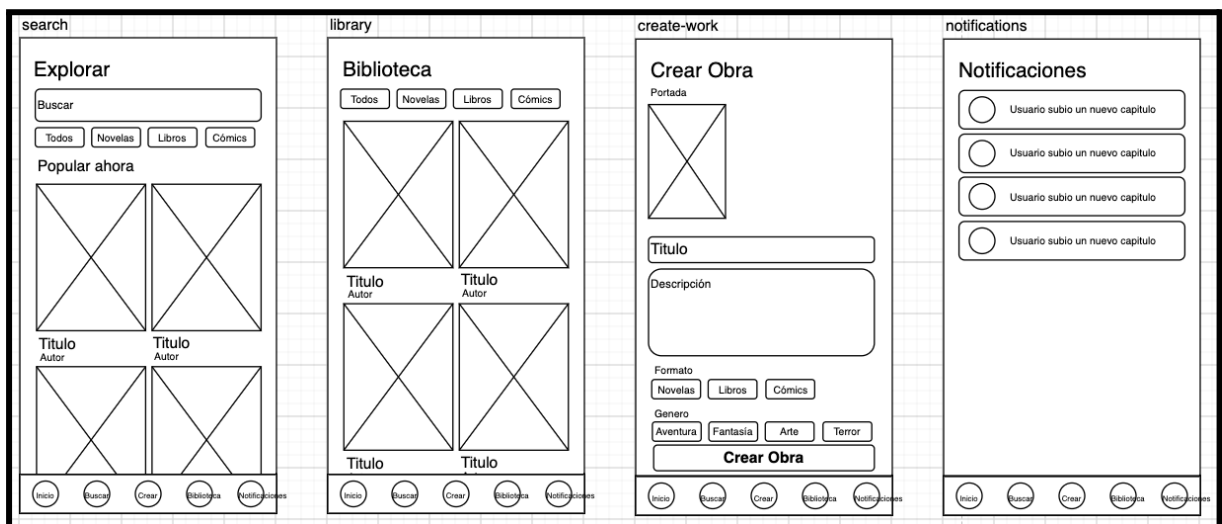
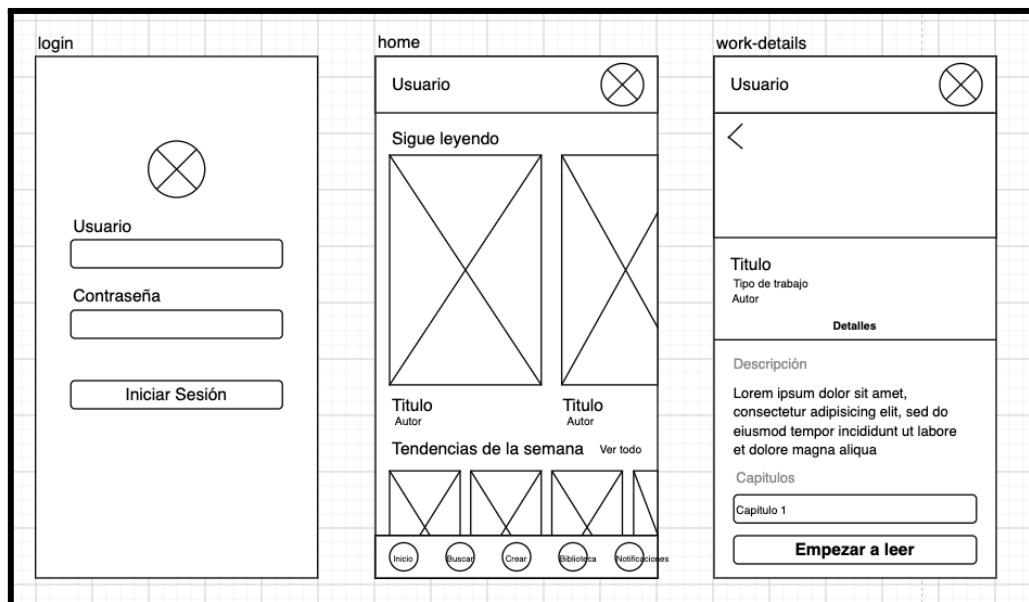
General:

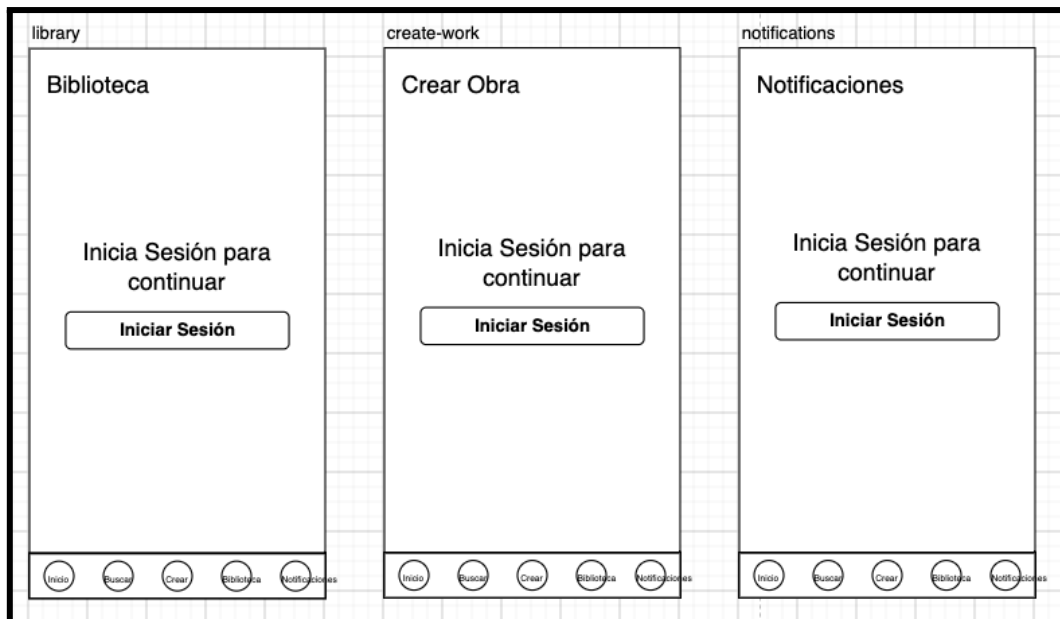
- Facilidad de uso
- Diseño y funcionamiento inclusivo
- Interfaz intuitiva y atractiva
- Actualización de cambios en tiempo real
- Retroalimentación inmediata de las acciones

● Diseño

● Boceto / Wireframes

- **login** - Vista de acceso a la cuenta
- **home** - Vista principal donde se muestra la galería de títulos y tendencias.
- **work-details** - Muestra los detalles de la obra, se abre al seleccionar una.
- **search** - Permite buscar obras
- **library** - Vista de obras guardadas para leer de un usuario
- **notifications** - Vista de las notificaciones de capítulos nuevos o demás
- **create-work** - Permite crear una obra nueva
- **register** - Vista de registro de cuenta

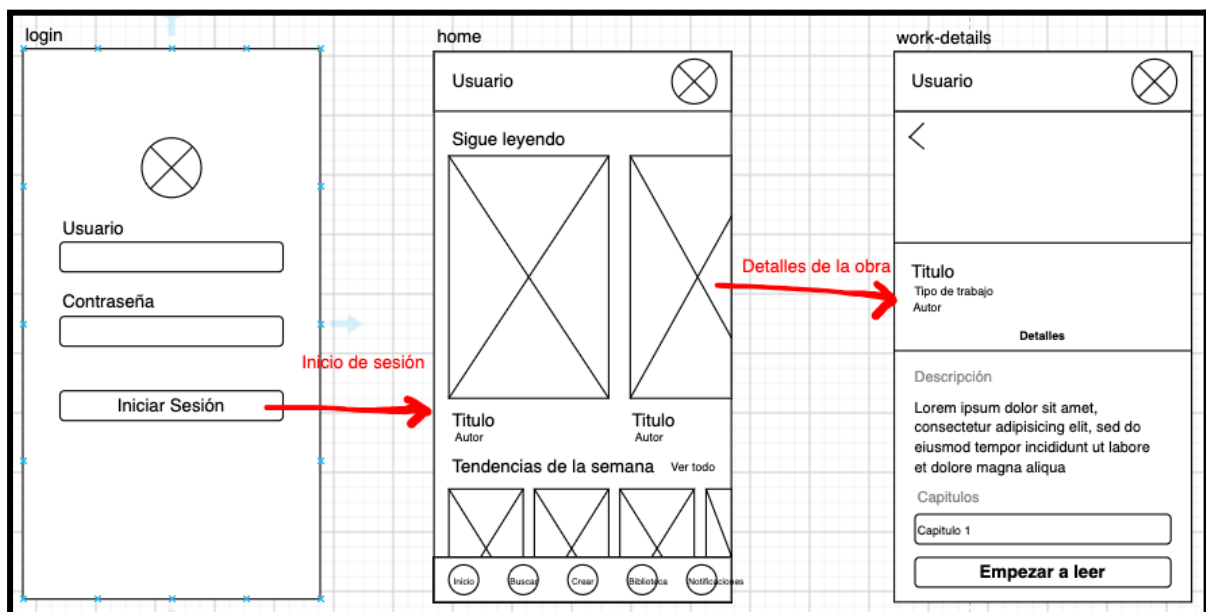




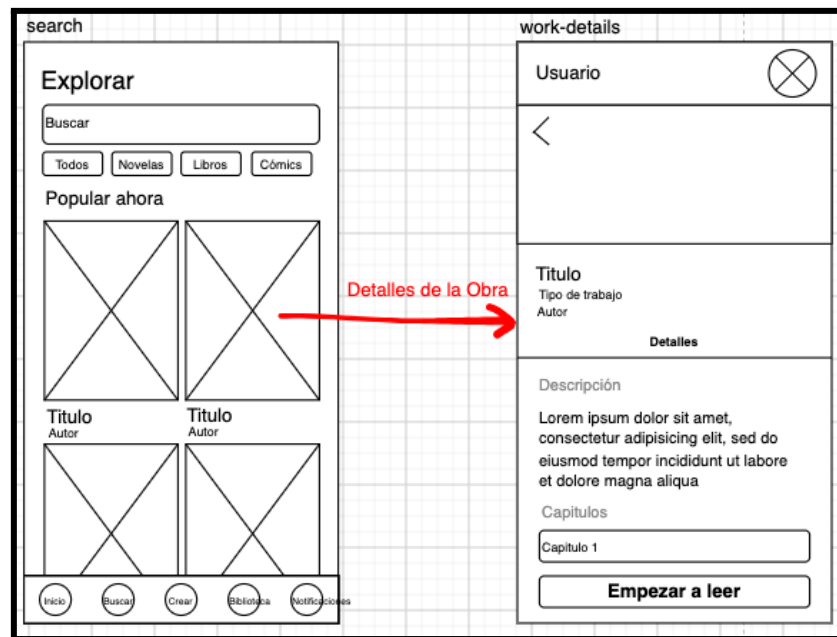
- **Flujo de usuarios**

Flujo de navegación básico

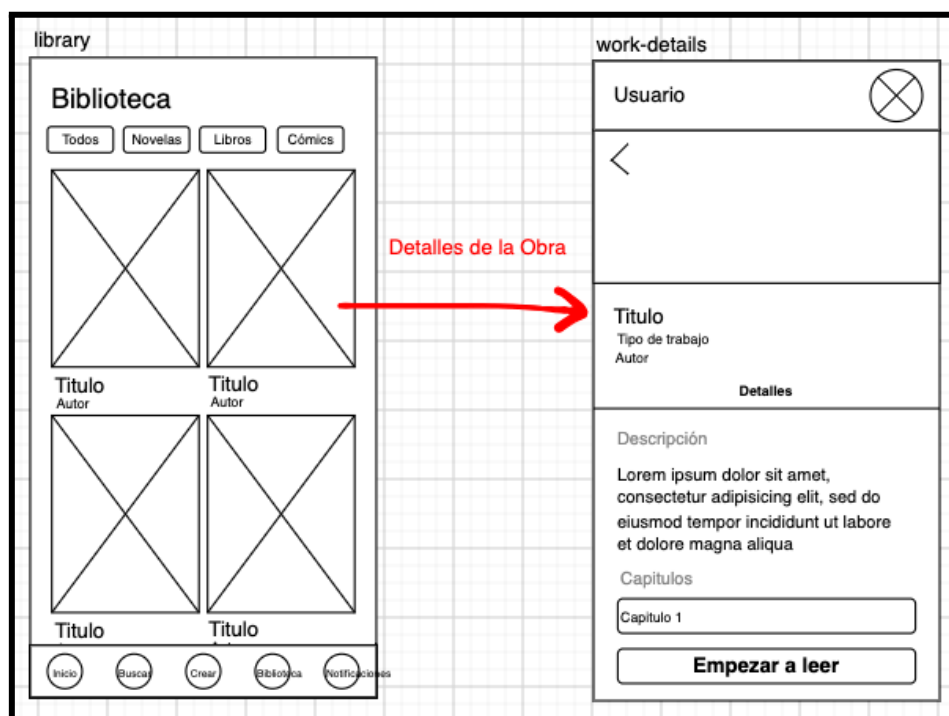
Inicio de Sesión -> Home -> Detalles de obra



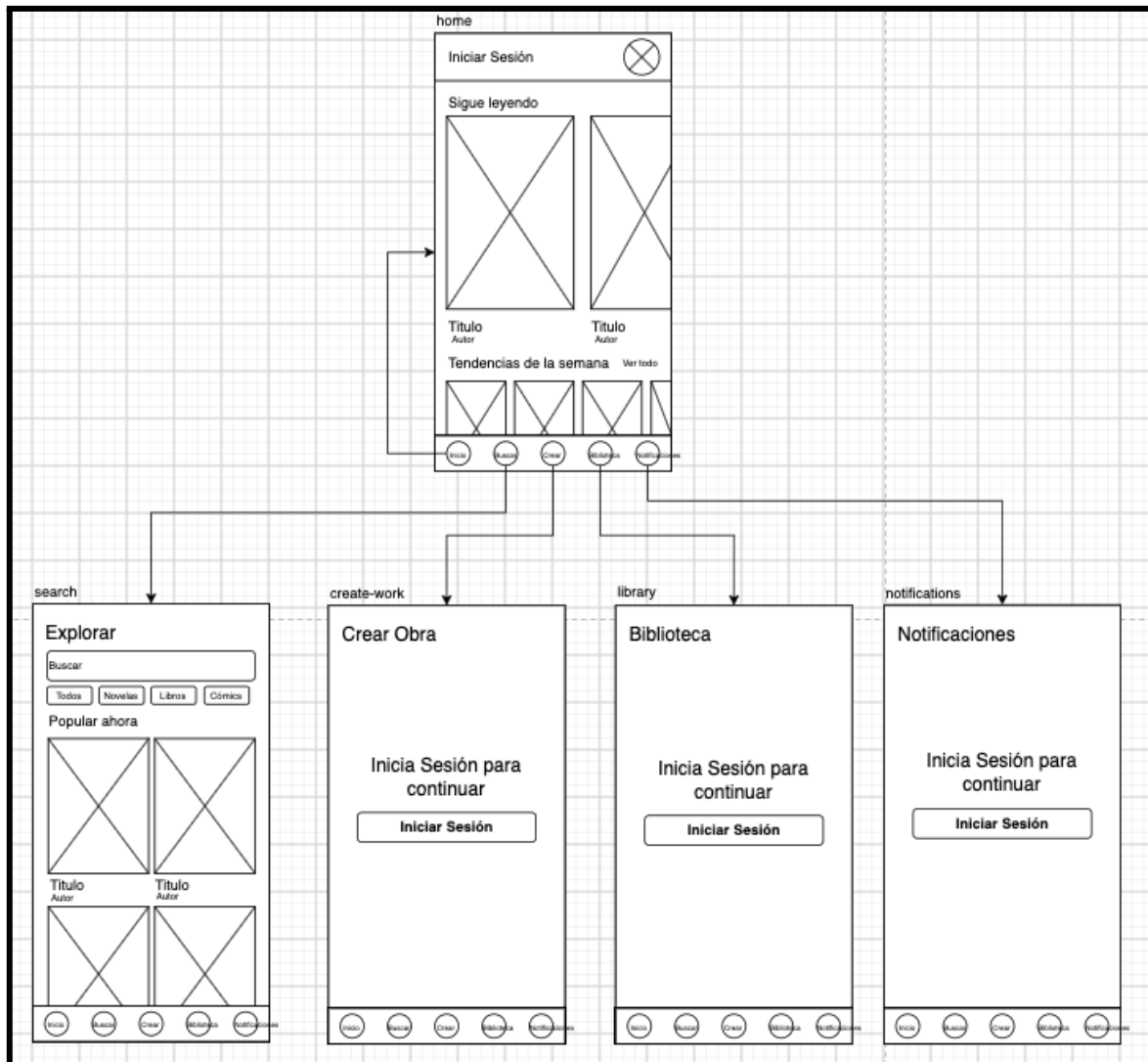
Explorar obras -> Detalles de Obras



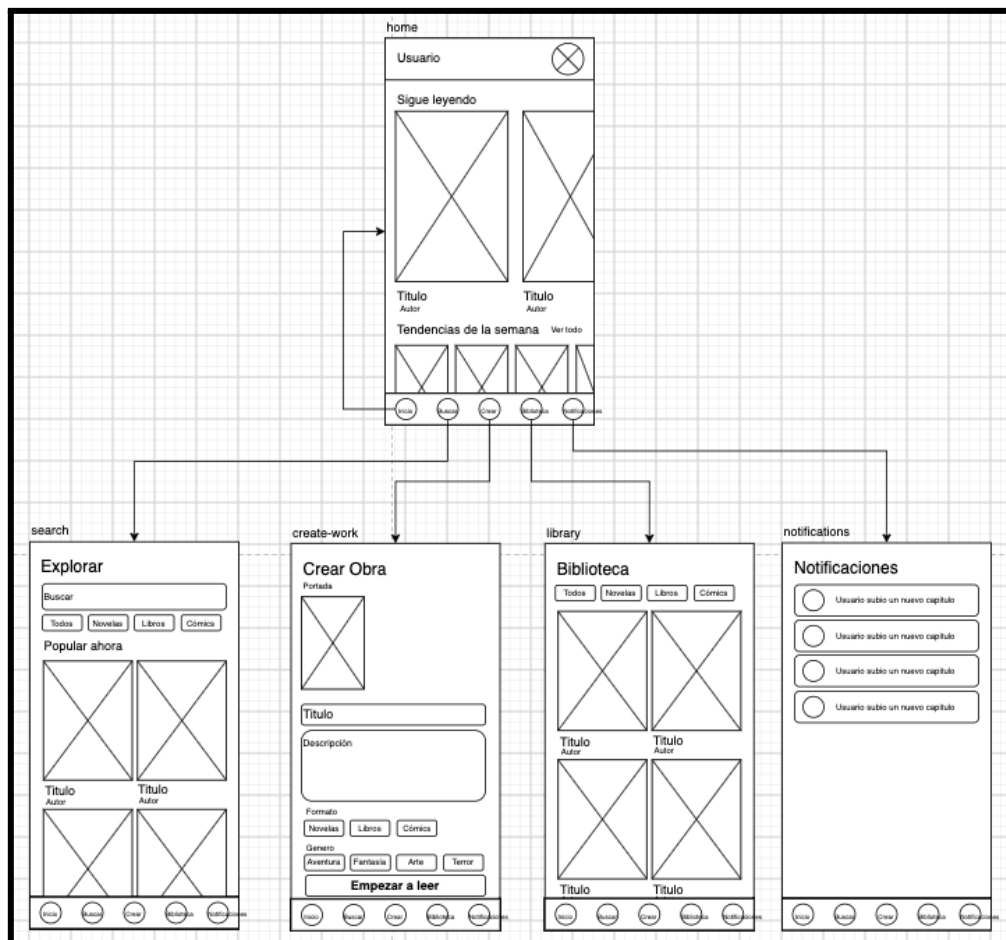
Biblioteca -> Detalles de Obra



Usuario no ingresado



Usuario ingresado



● Identidad visual

Se tomaron en cuenta los siguientes elementos para representar la identidad visual de la aplicación:

- **Elementos que recuerdan el proceso de escritura:** plumas, papel, libros, etc.
- **Elementos que evoquen el proceso creativo:** líneas curvas, garabatos, escritura y dibujo.
- Colores cómodos, intensos y vibrantes.
- Creatividad, pasión, importancia, formalidad editorial.

Elementos representativos:

- Pluma de ave
- Arte vectorial de líneas continuas
- Figuras: Libros, avioncitos de papel, elementos de escritura, etc.
- Color negro y colores vibrantes que representen la pantalla de los medios digitales.

- **Paleta de colores**

Elección funcional:

- **Onyx (#131214) y Shadow Grey (#282728):** proveen una base sólida y neutra. Son ideales para fondos y tipografía principal, porque reducen la fatiga visual y permiten que las imágenes o textos destaquen sin distracciones.
- **Ash Grey (#9EA099):** funciona como tono intermedio para separar secciones, resaltar subtítulos o crear jerarquía visual sin romper la armonía.
- **Honeydew (#F1FAEE):** aporta aire y luminosidad. Es perfecto para fondos de lectura prolongada, ya que suaviza el contraste y transmite calma.
- **Raspberry (#D43642):** el acento emocional. Sirve para botones, enlaces, llamadas a la acción o destacar citas importantes. Su energía rompe la neutralidad y guía la atención.

Estética narrativa:

La combinación de oscuros y claros crea un **ambiente editorial elegante**, cercano al diseño de revistas premium o plataformas literarias.

El contraste entre Honeydew y Raspberry introduce un **ritmo visual**: lo claro invita a leer, lo vibrante invita a interactuar.

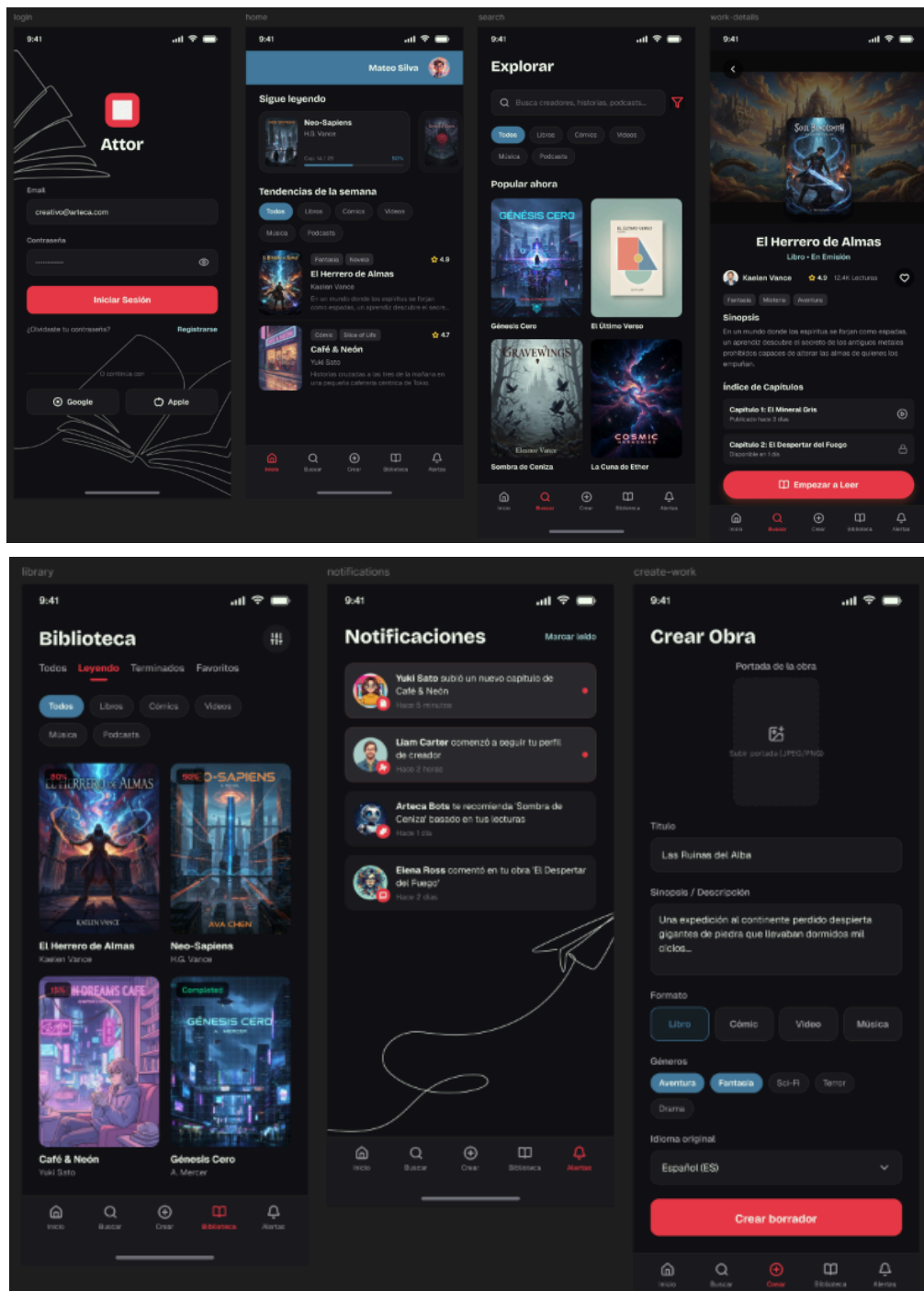
La paleta transmite **seriedad con un toque humano**: los grises evocan profesionalismo, mientras el rojo frambuesa añade pasión y vitalidad, cualidades esenciales en historias.

Usabilidad:

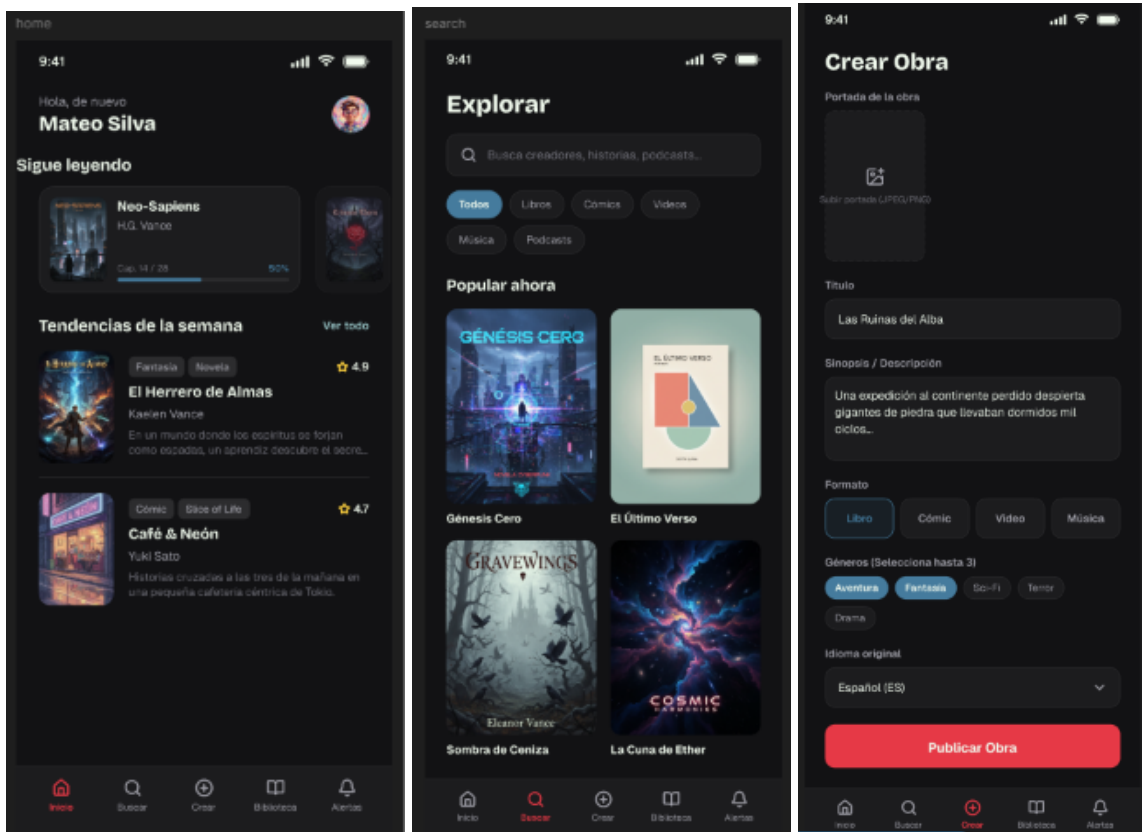
- **Accesibilidad:** el contraste entre Onyx/Honeydew y Raspberry cumple con estándares de legibilidad, asegurando que botones y textos sean visibles para usuarios con distintas capacidades visuales.
- **Escalabilidad:** la paleta es versátil; puede adaptarse tanto a modo oscuro como claro sin perder coherencia.
- **Consistencia:** al limitarse a cinco tonos, se evita la saturación cromática y se garantiza una identidad visual clara y reconocible.



- Prototipo inicial de Figma



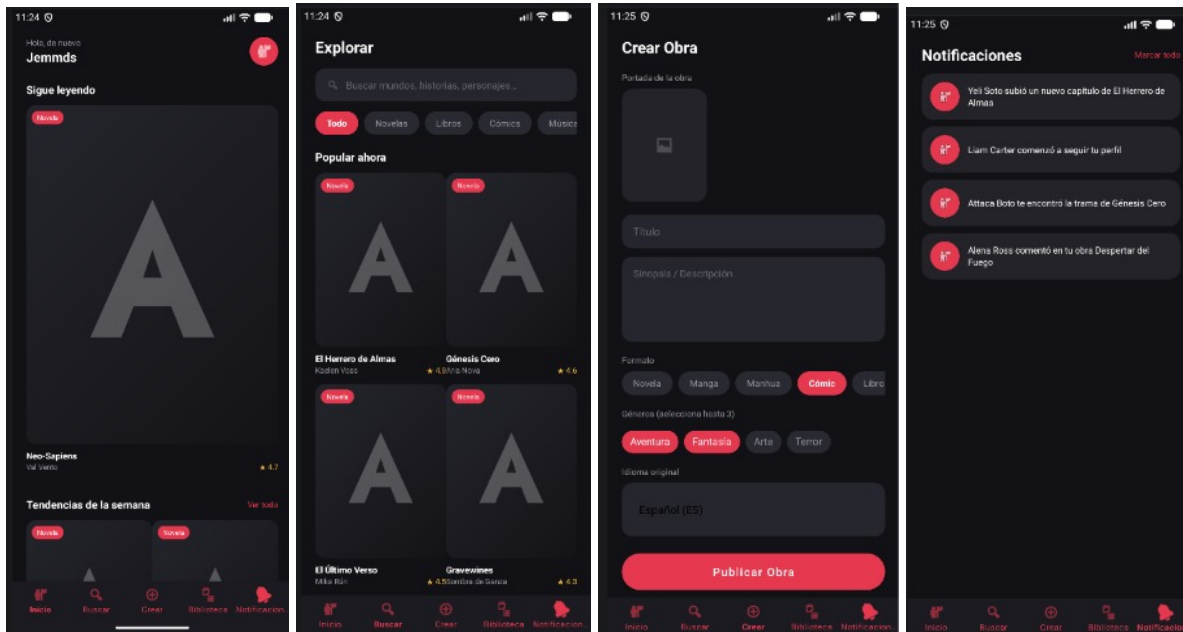
- **Modificaciones al prototipo de Figma**



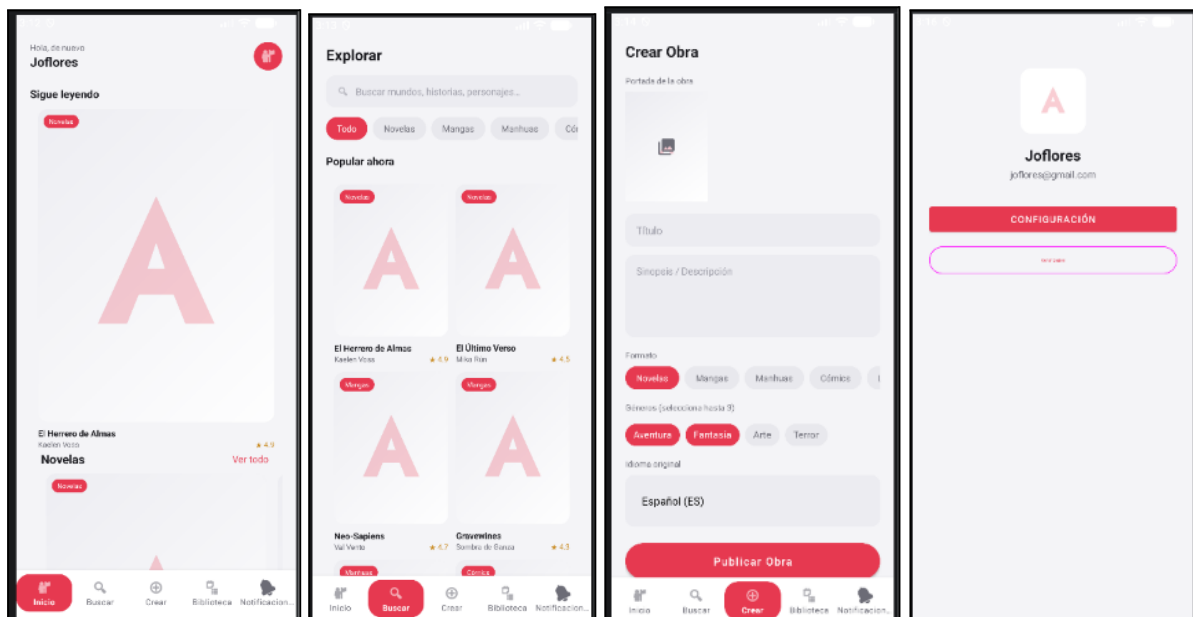
1. Se eliminó la barra fija inicial de bienvenida del usuario por una bienvenida más sutil y que pueda desaparecer al deslizar, para darle prioridad a mostrar las obras.
2. En el buscador de obras se consideró mejor cambiar el display de las categorías a un carrusel que se deslice y que ocupe menos espacio (considerando que en un futuro pueden aumentar o cambiar las categorías).
3. En la vista de obra se descartó implementar una imagen de fondo adicional a la portada. Se movió el marcador de “me gusta” a la esquina de arriba a la derecha.

Se implementaron más cambios en el diseño para añadir el modo “light” que puede implementarse desde la configuración de la configuración en el perfil. También en la página de inicio se consideró muy importante clasificar las recomendaciones de obras por carruseles según los formatos.

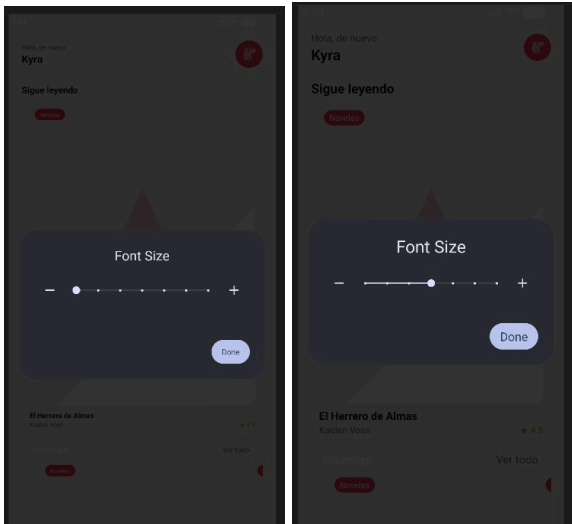
- **Primera versión del prototipo funcional en Android Studio**



- **Versiones avanzadas del prototipo en Android Studio**



Para mejorar la accesibilidad será necesario mejorar el contraste y aumentar el tamaño de la letra en “cerrar sesión” además de confirmar que el tamaño de letra aumenta con el sistema y que el tamaño de los botones es adecuado (mínimo 8 px de padding):



También se probó la paleta de colores con distintos filtros de daltonismo, asegurando que se mantenga un contraste suficiente entre los colores:

131214	282728	9EA099	F1FAEE	D43642	Color blindness ☒ Protanopia ☐ Deuteranopia ☐ Tritanopia ☐ Achromatopsia
121213	272727	9E9E9A	F4F4F0	8F8E3F	
131214	282728	9EA099	F1FAEE	D43642	Color blindness ☐ Protanopia ☐ Deuteranopia ☒ Tritanopia ☐ Achromatopsia
121313	272727	9E9C9C	F1F3F3	CC3C3C	
131214	282728	9EA099	F1FAEE	D43642	Color blindness ☐ Protanopia ☐ Deuteranopia ☐ Tritanopia ☒ Achromatopsia
121212	272727	9E9E9E	F5F5F5	666666	

● Funcionalidades

Se manejan dos tipos de roles:

1. Invitado
2. Usuario registrado

Vista de invitado:

- Muestra el inicio, el apartado de búsqueda y el apartado de biblioteca
- Muestra que es cuenta de invitado
- Se puede entrar y leer las obras
- Cuando se quiere ingresar a la biblioteca de obras guardadas solicita iniciar sesión.

Inicio de sesión:

- Se inicia sesión con correo electrónico y contraseña
- Al ingresar caracteres en el apartado de contraseña se ocultan los caracteres y únicamente se muestra el último ingresado por un segundo.

Pantalla de inicio:

- Al hacer clic en una de las obras te lleva a la vista de detalles de la obra para poder leerla.
- Muestra y recomienda obras en un grid de carruseles clasificados por categorías.
- Muestra el usuario o “invitado” en la parte superior.

Creación de obras:

- La opción aparece únicamente con una cuenta registrada.
- Permite seleccionar el formato y los géneros de la obra.
- Permite seleccionar el idioma en que está la obra desplegando una lista.
- La nueva obra creada se agrega al listado de obras disponibles.

Búsqueda:

- El selector de categorías si filtra las obras.
- El buscador de obras funciona y arroja los resultados buscados. No hace distinción de letras mayúsculas y minúsculas.

Barra de navegación inferior:

- Con los usuarios con sesión iniciada se muestran los apartados “inicio”, “buscar”, “crear”, “biblioteca” y “notificaciones”.

Biblioteca:

- Permite el acceso únicamente con una cuenta registrada.
- Muestra un grid con las obras guardadas.

Notificaciones:

- Permite el acceso únicamente con una cuenta registrada.
- Muestra las notificaciones de las obras, comentarios y autores.

● Implementación de accesibilidad e inclusión

Se tomaron varias consideraciones durante el desarrollo donde nos consultamos qué dificultades teníamos para usarla y realizamos ajustes en los tamaños de los botones, en el contraste de los elementos, entre otros detalles que se mencionarán:

Las imágenes decorativas que no aportan información (como el fondo de una portada o un ícono decorativo) se marcan como "no importantes" para que el lector de pantalla las ignore y no las lea. sí por favor

También se ajustaron los colores de texto, íconos y estrellas para que cumplan con las pautas WCAG (Web Content Accessibility Guidelines). Esto significa que el texto tiene suficiente contraste con el fondo para que personas con baja visión puedan leerlo sin dificultad.

La app permite al usuario cambiar el tamaño de letra desde la configuración del dispositivo (las capturas previas en el documento muestran justo esta parte). El valor se guarda y se aplica a toda la app, lo que ayuda a personas con dificultades visuales.

Tiene un tema oscuro completo que evita la fatiga visual en ambientes con poca luz. Se puede activar manualmente o seguir la configuración del sistema.

También crearon layouts especiales según la orientación y la responsividad, de modo que la app se vea bien y sea usable en cualquier dispositivo, lo que facilita la experiencia de los usuarios. Entre otras funcionalidades probadas como el filtro de daltonismo.

- **Implementación de interacciones, validaciones y manejo de eventos**

- **Marcar como me gusta una obra (estrella):** cuando el usuario toca la estrella en la pantalla de detalles, la app guarda ese "me gusta" en el dispositivo. La estrella cambia de vacía a llena y el rating de la obra sube o baja inmediatamente. Si el usuario cierra y vuelve a abrir la app, el "me gusta" sigue guardado.
- **Validación de datos al crear obra:** antes de publicar, la app revisa que el usuario haya llenado el título, la sinopsis y el formato. Si falta algo, no deja publicar. Cuando todo está completo, muestra un mensaje de éxito con animación.
- **Validación de datos al iniciar sesión:** la app verifica que el email y la contraseña no estén vacíos. Luego compara la contraseña ingresada con la que está guardada en la base de datos (usando un hash de seguridad).
- **Validación de datos al registrar usuario:** antes de crear una cuenta, la app revisa que ese email no esté ya registrado. Si ya existe, muestra un mensaje de error.
- **Al crear una obra nueva se puede seleccionar:** el formato (novela, manga, cómic, etc. con botones tipo chips), los géneros (se pueden elegir hasta 3) y el idioma (en un menú desplegable con 5 opciones).
- **Navegación:** la app tiene una barra inferior con 5 pestañas (Inicio, Buscar, Crear, Biblioteca, Notificaciones). Al cambiar de pestaña, la pantalla se desvanece y aparece la nueva con una animación suave. Las otras pantallas (Perfil, Configuración, Detalles de obra) se abren como ventanas nuevas.
- **Eventos de toque:** al tocar una tarjeta de obra se ven sus detalles, al tocar el avatar se va al perfil, al tocar "Ver todo" se ven todas las obras de esa categoría, y al tocar un capítulo se reproduce (aunque todavía es un prototipo).

- **Tecnologías y herramientas utilizadas**

- **Lenguaje:** Kotlin 1.9.24
- **Entorno de desarrollo:** Android Studio con Gradle 8.5.0 y Android Gradle Plugin 8.5.0
- **Base de datos local:** Room 2.6.1 con KSP (Kotlin Symbol Processing) para compilación de entidades y DAOs
- **Interfaz de usuario:** Material Components 1.12.0, ConstraintLayout 2.1.4, RecyclerView 1.3.2, CardView 1.0.0

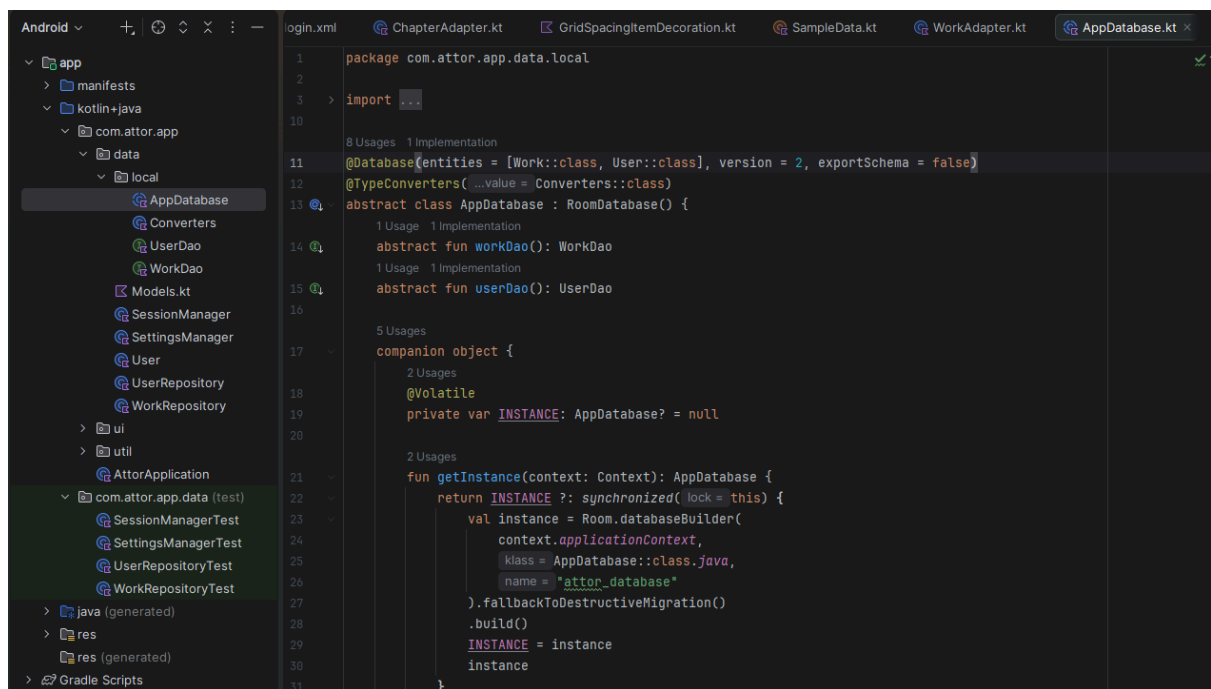
- **Programación asíncrona:** Kotlin Coroutines 1.8.1 con lifecycleScope para operaciones de base de datos
- **Fragmentos:** Fragment KTX 1.8.1 con transacciones de fragmentos y animaciones personalizadas
- **Persistencia de sesión y configuración:** SharedPreferences (SessionManager y SettingsManager)
- **Seguridad:** hash SHA-256, que es un algoritmo que convierte la contraseña en un código irreversible para no guardarla en texto plano
- **Pruebas:** JUnit, Mockito y Robolectric, que son herramientas para probar que el código funciona correctamente sin necesidad de un teléfono físico
- **Versión mínima de Android:** 7.0 (API 24), versión objetivo: Android 14 (API 34)

● Implementación de la persistencia de datos

La persistencia se implementa mediante Room, la librería de abstracción de SQLite de Android. Guardamos los datos ahí y nos aseguramos que los registros de libros y usuarios no se pierdan incluso al reiniciar la aplicación.

Se utilizan las entidades en lugar de los objetos para representar las tablas de la base de datos, hay una tabla de usuarios (con email, nombre, contraseña hasheada y fecha de creación) y una tabla de obras (con título, autor, rating, categoría, sinopsis, etc.).

Luego también se manejan los DAOs para realizar los queries/consultas a la base de datos, estos interactúan con la base de datos (que por default guarda 12 registros de obras que usamos de ejemplo).



```

1 package com.attor.app.data.local
2
3 > import ...
4
5
6
7
8 8 Usages 1 Implementation
9
10
11 @Database(entities = [Work::class, User::class], version = 2, exportSchema = false)
12 @TypeConverters(value = Converters::class)
13 abstract class AppDatabase : RoomDatabase() {
14
15     1 Usage 1 Implementation
16     abstract fun workDao(): WorkDao
17
18     1 Usage 1 Implementation
19     abstract fun userDao(): UserDao
20
21
22
23
24 5 Usages
25 companion object {
26
27     2 Usages
28     @Volatile
29     private var INSTANCE: AppDatabase? = null
30
31
32     2 Usages
33     fun getInstance(context: Context): AppDatabase {
34         return INSTANCE?.synchronized(lock = this) {
35             val instance = Room.databaseBuilder(
36                 context.applicationContext,
37                 class = AppDatabase::class.java,
38                 name = "attor_database"
39             ).fallbackToDestructiveMigration()
40             .build()
41             INSTANCE = instance
42             instance
43         }
44     }
45 }

```

This screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes folders for manifests, kotlin+java, and data, with sub-folders like local, ui, and util. The code editor displays the `UserDao` interface in the `com.attor.app.data.local` package. The interface defines three methods: `insertUser`, `getUserByEmail`, and `countByEmail`, each with its respective annotations and SQL queries.

```
1 package com.attor.app.data.local
2
3 > import ...
4
5 5 Usages 1 Implementation
6
7 @Dao
8
9
10 interface UserDao {
11
12     1 Usage 1 Implementation
13     @Insert(onConflict = OnConflictStrategy.ABORT)
14     suspend fun insertUser(user: User)
15
16     1 Usage 1 Implementation
17     @Query(value = "SELECT * FROM users WHERE email = :email LIMIT 1")
18     suspend fun getUserByEmail(email: String): User?
19
20     1 Usage 1 Implementation
21     @Query(value = "SELECT COUNT(*) FROM users WHERE email = :email")
22     suspend fun countByEmail(email: String): Int
23 }
```

This screenshot shows the same IDE with the `WorkDao` interface selected in the project structure. The code editor displays the `WorkDao` interface in the `com.attor.app.data.local` package. The interface defines four methods: `getAllWorks`, `getWorkById`, `insertWorks`, and `insertWork`, each with its respective annotations and SQL queries.

```
1 package com.attor.app.data.local
2
3 > import ...
4
5 5 Usages 1 Implementation
6
7 @Dao
8
9
10 interface WorkDao {
11
12     1 Usage 1 Implementation
13     @Query(value = "SELECT * FROM works")
14     suspend fun getAllWorks(): List<Work>
15
16     1 Usage 1 Implementation
17     @Query(value = "SELECT * FROM works WHERE id = :id")
18     suspend fun getWorkById(id: String): Work?
19
20     1 Usage 1 Implementation
21     @Insert(onConflict = OnConflictStrategy.REPLACE)
22     suspend fun insertWorks(works: List<Work>)
23
24     1 Usage 1 Implementation
25     @Insert(onConflict = OnConflictStrategy.REPLACE)
26     suspend fun insertWork(work: Work)
27
28     1 Usage 1 Implementation
29     @Query(value = "SELECT COUNT(*) FROM works")
30     suspend fun getCount(): Int
31 }
```

This screenshot shows the same IDE with the `WorkRepository` class selected in the project structure. The code editor displays the `WorkRepository` class in the `com.attor.app.data` package. The class implements the `WorkDao` interface by using the `AppDatabase` to get an instance of `WorkDao` and then calling its methods.

```
1 package com.attor.app.data
2
3 > import ...
4
5 20 Usages
6
7 class WorkRepository(context: Context) {
8
9     5 Usages
10     private val dao = AppDatabase.getInstance(context).workDao()
11
12     10 Usages
13     suspend fun getAll(): List<Work> {
14         return dao.getAllWorks()
15     }
16
17     4 Usages
18     suspend fun getById(id: String): Work? {
19         return dao.getWorkById(id)
20     }
21
22     2 Usages
23     suspend fun publish(work: Work) {
24         dao.insertWork(work)
25     }
26 }
```

● Pruebas unitarias

Las pruebas verifican la lógica de negocio sin necesidad de un dispositivo:

- **UserRepositoryTest** (6 pruebas): registro de usuario, email duplicado, login con credenciales correctas/incorrectas, y que la contraseña se guarde hasheada (SHA-256).
- **WorkRepositoryTest** (7 pruebas): siembra de datos, no duplicar al sembrar, obtener todas las obras, buscar por ID existente/inexistente, publicar obra nueva, y verificar que los formatos sean correctos.
- **SessionManagerTest** (9 pruebas): estado por defecto (invitado), login con usuario, login como invitado, logout, y que los datos persistan correctamente.
- **SettingsManagerTest** (6 pruebas): valores por defecto (modo noche seguir sistema, escala 1.0), y que los cambios de modo oscuro/claro y tamaño de letra persistan.

Todas usan Robolectric para simular el contexto de Android y coroutines para las funciones suspend.

The screenshot displays the Android Studio interface. The top toolbar shows icons for running tests. The left sidebar shows the project structure with the following folders: `app`, `manifests`, `kotlin+java`, `com.attor.app`, `com.attor.app.data (test)`, `com.attor.app.data.local`, `res`, and `res (generated)`. The `com.attor.app.data (test)` folder is expanded, showing `SessionManagerTest`, `SettingsManagerTest`, `UserRepositoryTest`, and `WorkRepositoryTest`. The main editor shows the `SessionManagerTest.kt` file with the following code:

```
1 package com.attor.app.data
2
3 import ...
4
5 @RunWith(value = RobolectricTestRunner::class)
6 @Config(sdk = [34])
7 class SessionManagerTest {
8
9     2 Usages
10     private lateinit var context: Context
11     24 Usages
12     private lateinit var sessionManager: SessionManager
13
14     @Before
15     fun setup() {
16         context = ApplicationProvider.getApplicationContext()
17         sessionManager = SessionManager(context)
18         sessionManager.logout()
19     }
20
21     @Test
22     fun isGuest_defaultState_returnsTrue() {
23         assertTrue(sessionManager.isGuest())
24     }
25
26     @Test
27     fun isLoggedIn_defaultState_returnsFalse() {
28         assertFalse(sessionManager.isLoggedIn())
29     }
30
31     @Test
32     fun getUserName_defaultState_returnsInvitado() {
33         assertEquals(expected = "Invitado", actual = sessionManager.getUserName())
34     }
35 }
```

The bottom of the screenshot shows three test result panels. The first panel shows the test `isGuest_defaultState_returnsTrue` passing in 41 seconds. The second panel shows the test `isLoggedIn_defaultState_returnsFalse` passing in 3 seconds. The third panel shows the test `getUserName_defaultState_returnsInvitado` passing in 3 seconds.

- **Uso de IA**

Se usó **Claude** IA para poder plasmar el diseño en el programa, para así hacer la parte visual del código con archivos .xml y hacer el diseño a futuro también para que se pueda hacer funcional y escalable, para evitar problemas de versiones futuras en el código, ya sea las conexiones de pantallas con los botones, el llamado de las variables para poder usarlas en toda la aplicación y el uso del almacenamiento del dispositivo para que los usuarios no necesiten iniciar sesión constantemente dentro de la app.

También se tomó el apoyo del modelo “LongCat 2.5” en open code para la revisión y análisis del código del proyecto, recomendaciones, la implementación de pruebas a la aplicación y el apoyo para la identificación de las características para la documentación efectiva de cada parte.